

Computational Models for Sustainable Chemical Systems

In this lecture block we will study how computational models support the development of sustainable chemical systems with a particular focus on data-driven models. Together we will:

- dive into the foundations of modelling
- develop data-driven models with hands-on examples (toxicity and solubility predictions)
- apply the models for selecting solvents in line with green chemistry principles

General information

Lecturer

Dr.-Ing. Laura König-Mattern Group Leader for Computational Biorefining

Center for the Transformation of Chemistry

Mail: laura.koenig-mattern@ctc-germany.org

Homepage: <https://transforming-chemistry.org/forschung/forschungsgruppen/#biorefineries>



- background in **Process Engineering**
 - Bachelor in **Biosystems Engineering** (Otto von Guericke University Magdeburg)
 - Master in **Systems Engineering and Engineering Cybernetics** (Otto von Guericke University Magdeburg)
- PhD and research on **computer-aided molecular and process design for biorefineries** at Max Planck Institute for Dynamics of Complex Technical Systems
- Guest PhD at EPFL in Lausanne in the the **Sustainable and Catalytic Processing** Lab

Course materials

You can find all course materials on GitHub: [Sustainable Chemical Systems Course](#)

- Our lectures are a mix of theory and hands-on coding (Python). Therefore, the course contents are implemented in a jupyter notebook that you can run locally on your machine or simply open it in Google colab (Installation instructions see below)
- HTMLs/PDFs: If you rather prefer htmls or pdfs, check out the "html_export" and "pdf_export"-folders. The pdfs are perfect to take notes on any device you like.

Installation

Find all installation instructions [here](#)

Literature

Books:

- **Statistics for Chemical Engineers: From Data to Models to Decisions** by V. M. Zavalá : Awesome book about modelling and machine learning in chemical engineering.
- **Deep Learning** by I. Goodfellow, Y. Bengio and A. Courville : Open access, authored by pioneers on deep learning.
- **Hands-On Machine Learning with Scikit-Learn and TensorFlow** by A. Geron : For people who prefer learning by doing.

Online Courses:

- [Deep Learning Specialization](#) by Andrew Ng, K. Katanforoosh and B. Mourri from Coursera learning platform: A course series that requires a minimum of mathematical skills but still brings you from zero to a very good understanding and practical training on deep learning.
- [Machine Learning for Chemical Engineering](#) by E. A. del Rio Chanona : New course from the OptiML PSE Lab @ Imperial College London, I highly recommend to stay tuned!

Note

The code we develop is optimized for educational purposes rather than for performance. I tried to keep it as simple and yet informative as possible to make it accessible to various level of coding experience.

Colab compatibility

To make the notebook compatible with Google colab, we need to make some adjustments to the paths. The code snippet detects whether we run the notebook in colab or on our local machine. Then, we define the paths depending on this:

Course outline

Our lectures will evolve around the following practical example: **Identifying "green" solvents for lignin dissolution**

Lignin dissolution is an important step in **extracting lignin from biomass** but also for **lignin processing** into bio-based materials such as lignin-based coatings, films and nanoparticles. Therefore, solvents are commonly involved in many process units starting from extraction, followed by separation, up to the final product.

Utilizing lignin is sometimes directly dubbed as "green". However, the solvent choice is a design decision that is made on the molecular level with far reaching implications for its impact on environment, safety, and energy demand on the overall process system. Still today, solvent selection is often guided by intuition and experience. Computational models can accelerate the search and contribute to a rational solvent choice based on a systematic decision process.

In this course block, we will:

- develop a data-driven model in Python for predicting acute toxicity for chemicals
- explore graph neural networks for predicting lignin solubility
- learn computational approaches for solvent selection in line with the principles of green chemistry
- perform a solvent screening to identify solvents in line with green chemistry principles

On the fly, we will dive into the fundamentals of modelling, machine learning and learn a lot about toxicity and lignin dissolution as well!

Problem formulation

The inverse problem

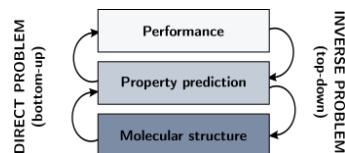
Direct problem: Given a defined molecular structure, predict one or more target property.

Inverse problem: Given one or more target properties, identify the molecular structure.

Our problem: We want to identify a solvent that:

- dissolves lignin well (maximizes lignin solubility)
- is in line with the principles of green chemistry (due to the limited time we have in our course, we will focus on acute toxicity only)

The search for the optimal solvent is therefore categorized as an **inverse problem**:



However, finding the optimal structure in the vast **chemical space** is a complex problem. The chemical space comprises all possible stable chemical compounds, defined by structural rules, encompassing known molecules, those synthetically feasible, and hypothetical ones. There are several estimates of the chemical space size depending on max. molecular size. In typical solvent size range (max. 17 heavy atoms), there are estimated to be 166 billion molecules! (Read more [here](#))

In the course example, we will develop a screening approach that ranks hundreds of solvents in a high-throughput fashion with respect to their lignin solubility and acute toxicity using a model-based approach.

Modelling goal

We want to develop a model that predicts **lignin solubility** and **acute toxicity**.

Recap: Principles of green chemistry

This lecture lays particular focus on how to facilitate lignin processing as a renewable resource using non-toxic solvents:

1. Waste prevention
2. Atom economy
3. Less Hazardous Chemical Syntheses
4. Designing Safer Chemicals
5. **Safer Solvents and Auxiliaries**
6. Design for Energy Efficiency
7. **Use of Renewable Feedstocks**
8. Reduce Derivatives
9. Catalysis
10. Design for Degradation
11. Real-time analysis for Pollution Prevention
12. Inherently Safer Chemistry for Accident Prevention

Target properties

Toxicity

"All things are poison, and nothing is without poison; the dosage alone makes it so a thing is not a poison." - Paracelsus, 1538

Depends on:

- Dose / concentration: The dose makes the poison
- Route of exposure (oral, inhalation, dermal)
- Duration (acute vs. chronic)
- Organism susceptibility (Human, rat, rabbit, mouse, algae, fish)

Definitions

Type	Description	Example
Acute toxicity	Effects from short-term exposure	Single high-dose poisoning
Chronic toxicity	Effects from long-term exposure	Carcinogenicity, liver damage
Subacute / chronic toxicity	Intermediate duration exposure	Repeated exposure for weeks/months
Specific organ toxicity	Targets specific organs	Hepatotoxicity (liver), nephrotoxicity (kidney)
Developmental / reproductive toxicity	Affects reproduction or fetal development	Teratogens
Mutagenicity / Genotoxicity	Alters DNA or causes mutations	Some chemicals cause cancer or DNA damage

Note: For a holistic toxicity assessment, we would need to analyze all different forms of toxicity. Due to the limited time we have in the lecture, we need to focus on one type: **Acute toxicity**

Acute toxicity: Globally Harmonised System (GHS) classification

GHS Category	LD ₅₀ (rat, oral) [mg/kg body weight]	Toxicity class name	Signal word
Category 1	≤ 5	Extremely toxic	Danger
Category 2	> 5 – 50	Highly toxic	Danger
Category 3	> 50 – 300	Toxic	Danger
Category 4	> 300 – 2,000	Harmful	Warning
Category 5*	> 2,000 – 5,000	May be harmful	—
Not classified	> 5,000	Practically non-toxic	—

- Category 5 is optional under GHS and not always implemented. The cut-off between "toxic" and "non-toxic" is usually after Category 4.

Experimental measurements

Acute toxicity is measured in the lab using standardized, tightly regulated bioassays, designed to quantify short-term adverse effects after a single exposure. It is usually measured as lethal dose (LD) in mg of substance per kg body weight that is necessary to kill a specific organism.

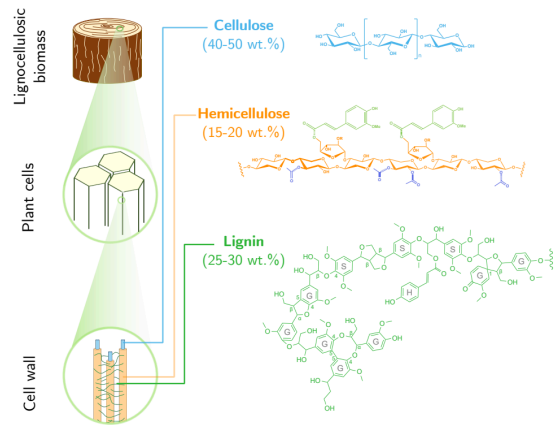
- LD₅₀: median lethal dose; dose that is required to kill half the members of a tested population after a specified test duration;
- Unit: mg substance per kg body weight
- Species: **Rats**, mice, rabbits
- Exposure routes:
 - Oral (most common)
 - Dermal
 - Intraperitoneal (abdominal cavity)

Note: Computational models not only support sustainable chemistry and chemical engineering, but also help to reduce animal tests

Lignin dissolution

Lignin structure

Recap:



from [König-Mattern, Dissertation, 2024](<https://www.bing.com/ck/a?!&p=ff5a887e437afc246faab6527b2bc835f166a63dcb53ab9e15861f7938c36939jmltdHM9MTc2NzgZMDQwMA&ptn=3&ver=2&hsh=4&fclid=033bdaf4-a3f6-6f2b-2e62-cc45a27f6e3c&psq=k%c3%b6nig+mattern+dissertation&u=a1aHR0cHM6Ly9yZXBvLmJpYmxpb3RoZWsudW5pLWhhbGxlLmRlL2JpdHN0cmVhbS8xOTgxMTg1OTIwLzExOTg1Ny8xL0slyzMIYjZuaWctTWFOdGVybl9MYXVyYV9EaXNzZXJ0YXRpb25fmjAyNC5wZGY>)

Lignin structure varies depending on:

- the feedstock (softwood, hardwood, grasses)
- the extraction method (e.g. Kraft process, Organosolv process, etc.)

Definition

$$w_{\text{lignin}} = \frac{m_{\text{lignin,dissolved}}}{m_{\text{solution}}} = \frac{m_{\text{lignin,dissolved}}}{m_{\text{lignin,dissolved}} + m_{\text{solvent}}}$$

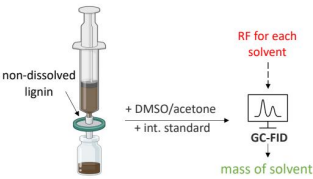
Experimental measurements

- weigh lignin into a known amount of solvent beyond expected solubility: saturated solution + remaining undissolved lignin.
- filter the suspension
- pipett a known amout of saturated lignin solution (m_{solution}) into vials and measure the amount of solvent in the solution using GC (m_{solvent})
- calculate the mass of lignin

Since

$$m_{\text{solution}} = m_{\text{lignin,dissolved}} + m_{\text{solvent}},$$

We can calculate $m_{\text{lignin,dissolved}}$ based the measured amount of solvent.



from [Chemical Engineering Journal](https://www.sciencedirect.com/science/article/pii/S2211339819300619), 'König-Mattern et al., Machine learning-supported solvent design for lignin-first biorefineries and lignin upgrading, CEJ, 2024.'

Model-based approaches to predict lignin solubility:

- Hansen solubility parameters (estimation)
- COSMO-RS (requires DFT calculations, slow)

Modelling

What is a model?

There are different definitions for "model" across disciplines

Philosophy

According to the philosopher Herbert Stachowiak, a model is characterized by at least three properties:

1. **Mapping**

A model always is a model of something. It is an image or representation of some natural or artificial, existing or imagined original, where this original itself could be a model.

2. **Reduction**

In general, a model will not include all attributes that describe the original but only those that appear relevant to the model's creator or user.

3. **Pragmatism**

A model does not relate unambiguously to its original. It is intended to work as a replacement for the original a) for certain subjects (for whom?) b) within a certain time range (when?) c) restricted to certain conceptual or physical actions (what for?).

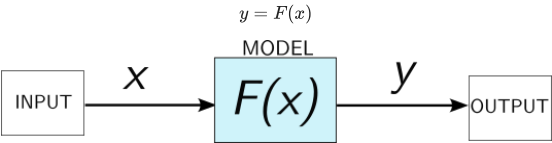
Mathematics

Mathematical models:

- are simplified descriptions of complex systems expressed in mathematical terms (parameters, variables, equations and inequalities, logical expressions)
- these mathematical terms can be manipulated
- simplifications are based on assumptions
- computers may help to solve these models

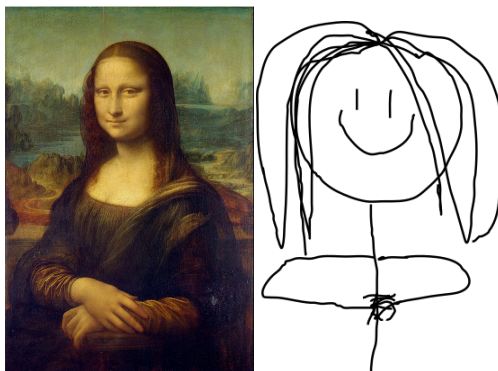
Mathematical modelling is the foundation for deriving models in other natural science and engineering disciplines, such as physics, chemistry, or chemical engineering.

A model is a simplified mathematical description of a complex system that maps inputs x to outputs y :



Art

The counterpart to a mathematical or physical model in art is abstraction:



left from [wikipedia](#), public domain

Why do we need models for sustainable chemical systems?

Useful for:

- gaining an in depth understanding about chemical systems on different scales (from the molecule up to the process system)
- optimizing reactions (space-time yield, conversion, selectivity, atom-efficiency, temperature, pressure, solvent consumption, catalyst loading)
- optimizing processes (reactor design, cost, energy, environmental impact)
- process design and scale-up
- process control
- safety and risk assesment
- life cycle assessment
- molecular screening and design: finding the optimal molecule for a certain application

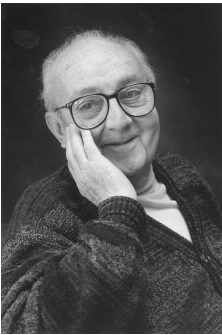
Modelling can make chemical systems more sustainble through optimizing molecules, reactions and processes for minimizing their environmental impact

What makes a model a "good model"?

Join at [menti.com](#), use code: 2188 5270



The famous British statistician George E. P. Box said the famous words:



from [wikipedia](https://en.wikipedia.org/wiki/George_E._P._Box#/media/File:GeorgeEPBox.jpg), public domain

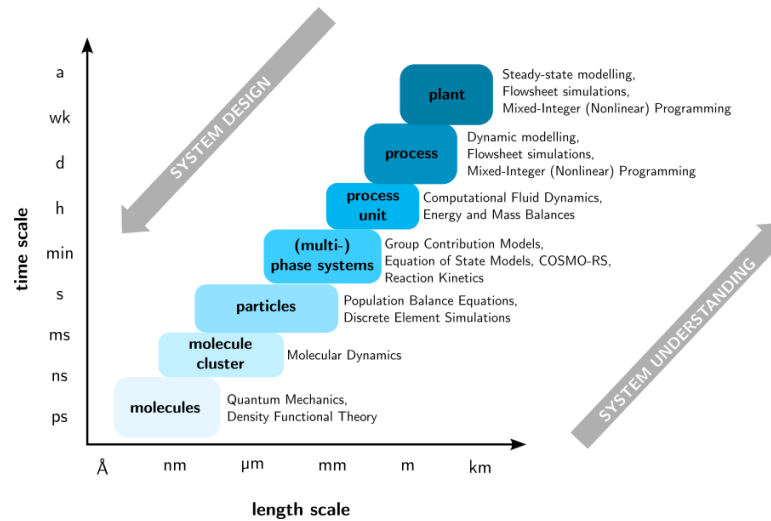
A good model:

- captures the essential behavior of the system
- answers the question at hand
- sufficient accuracy
- validated against experiments
- interpretable
- robust
- computationally feasible

Modelling across the scales

Choosing the right time- and length-scale is key to derive a "good model":

"All models are wrong, but some are useful."



Model types

Mechanistic vs. hybrid vs. data-driven

Mechanistic models:

A mechanistic model is **derived from known physical, chemical, or biological laws**, such as conservation laws (mass, energy, momentum), thermodynamics, reaction kinetics, transport phenomena. Since we know the underlying laws, we can explain the model outputs. Therefore, mechanistic models **encode causality explicitly**. **The model complexity is low**: Only a few equations are sufficient to model a complex systems.

Mechanistic models are therefore also known as "**white-box models**".

Strengths:

- extrapolates outside data
- interpretable & explainable
- respects physical laws

Weaknesses:

- requires mechanistic understanding of the system
- involve many parameters: estimation can be complicated

Data-driven models:

A data-driven model **learns input-output relationships only from data**. Physical, chemical or biological laws are not used to derive the model. Its internal logic and decision-making processes are mostly unknown or too complex for to understand. **The model complexity is high**: Data-driven models often rely on large model representations to approximate even simple functional relationships. **Causality is not explicitly encoded**: These models receive input data and produce output (make predictions or decisions), but do not provide proper explanations that can be interpreted easily for their outcomes.

Data-driven models are therefore also known as "**black-box models**".

Strengths (provides there is enough high-quality data):

- fast to build
- easily captures complex nonlinearities
- especially strong when underlying physical, chemicals or biological laws are unknown

Weaknesses:

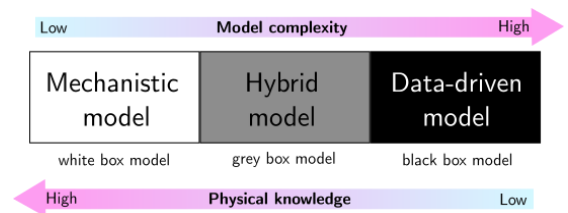
- heavily rely on vast amounts high-quality data
- poor extrapolation beyond trainign data
- limited interpretability
- can violate physical laws

Hybrid models:

A hybrid model **combines mechanistic and data-driven model approaches**. A mechanistic model structure can be enriched by terms derived from data-driven approaches. Or, a data-driven model can be enriched by mechanistic terms. In simple terms: Hybrid models use mechanistic approaches where we know the underlying laws and parameters are easy to measure, and data-driven approaches where the underlying laws are missing or too complex.

Examples:

- Residual ("error-correcting") hybrid models: Start with a mechanistic model and use data-driven approaches to capture what the mechanistic part misses.
- Parameter-learning models: A mechanistic model usually has many parameters that need to be determined to solve the model. Data-driven approaches can learn the parameters from data.
- Physics-informed neural networks (PINNs): PINNs are neural networks that learn from data while being constrained by physical laws.



There are also other model categories:

Linear vs. nonlinear models: A model is linear if it satisfies the superposition principle: scaling and addition of inputs lead to proportional scaling and addition of outputs. In simple terms: In a linear model, doubling the input exactly doubles the output, and the response to multiple inputs is the sum of the individual responses.

Dynamic vs steady-state models: A steady-state model neglects accumulation terms and describes only time-invariant system behavior, whereas a dynamic model includes accumulation terms and explicitly resolves temporal evolution. Therefore, a model is steady-state if time does not explicitly appear and all state variables are constant in time.

Discrete vs. continuous: A continuous model describes system evolution over a continuous independent variable, whereas a discrete model describes state changes at discrete points or events. For example: A model is discrete when it only gives predictions for certain points in times or space.

Data-driven modelling

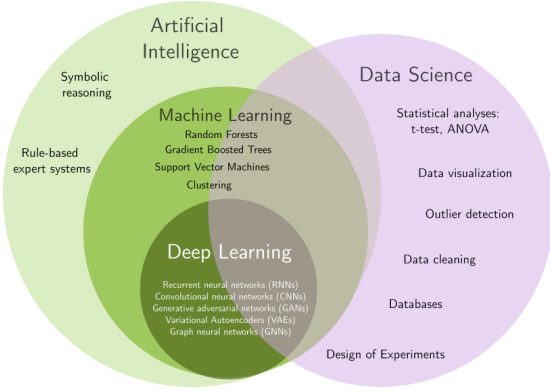
In our lecture series, we will have particular focus on data-driven models.

Definitions

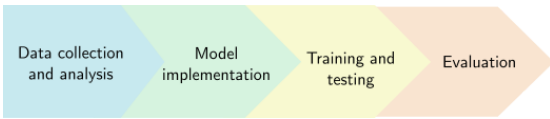
Artificial Intelligence (AI): Machines created with the aim to mimic human intelligence and human cognitive functions like problem-solving and learning. Can be rule-based (expert systems) or learning-based (ML).

Machine Learning (ML): ML models automatically learn patterns from data and improve their performance on tasks without explicit programming for every rule. ML models are typically trained on historical data by adjusting parameters to minimize prediction error or maximize performance metrics, and are evaluated on their ability to generalize to new, unseen data. They are typically used for prediction, classification, or pattern discovery.

Deep Learning (DL): Deep learning is a subset of machine learning that uses multi-layered artificial neural networks, inspired by the human brain. These networks consist of multiple layers of neurons that are interconnected. DL-based models automatically learn complex patterns and features from vast amounts of raw data, enabling advanced tasks like image recognition or natural language processing without explicit programming.



Deriving a data-driven model for predicting the acute toxicity of chemicals



Data collection and analysis

We will use the [CATMoS dataset](#):

- source: Mansouri et al., Environ Health Perspect, 2021 ;129(4):047013. doi: 10.1289/EHP8495
- CATMoS: Abbreviation for Collaborative Acute Toxicity Modeling Suite
- The dataset for acute toxicity (ortal, rat) is can be found [here](#)

Let's write some code to take a deeper look into the dataset!

In Python, we first import packages that we need for the code.

We will mostly use:

- Numpy: support for large, multi-dimensional arrays and matrices, comprehensive mathematical functions, random number generators, linear algebra routines, Fourier transforms, and more.
- Pandas: pandas is a data analysis and manipulation tool
- RDKit: cheminformatics package for deriving molecular features, plotting chemical structures and more
- Scikit-learn, Pytorch and Pytorch geometric: packages for machine learning
- tsne: dimensionality reduction

- matplotlib, seaborn: Packages for plotting:

```

7301
  Unnamed: 0      SMILES      val
0           0      Br/C=C\Br      62.0
1           1  BrC(Br)(Br)c1ccc2ccccc2n1  6769.0
2           2      BrC(Br)Br      933.0
3           3  BrC(Br)C(Br)Br     1200.0
4           4      BrC/C=C/CBr      62.0
...         ...
7296        7296      c1coc(OC2C02)c1     1100.0
7297        7297  c1nc(NC2CC2)c2ncn(C3CC3)c2n1     130.0
7298        7298  c1nc(NC2CC2)c2ncn(CC3CC3)c2n1     160.0
7299        7299      c1nc[nH]n1     1648.0
7300        7300      c1nnc(NCn2nncs2)s1     260.0

```

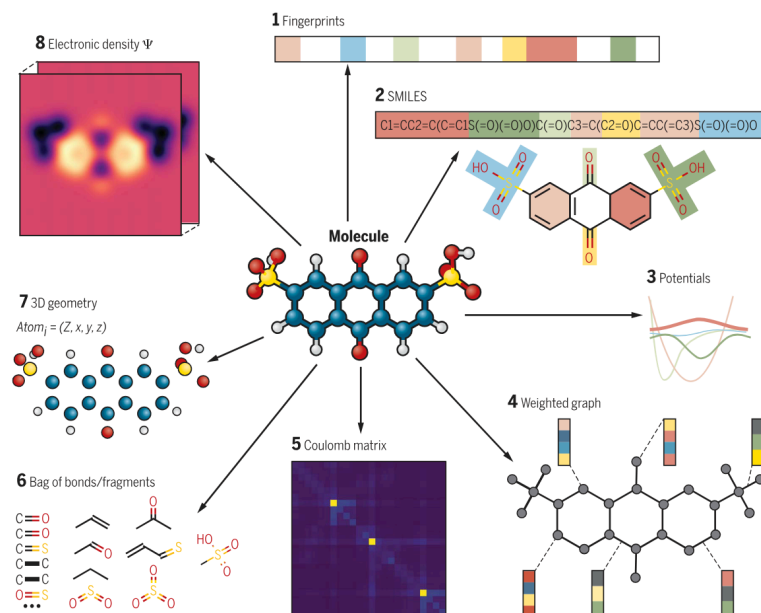
[7301 rows x 3 columns]

The dataset has two columns:

- **SMILES** : the molecular structures encoded as SMILES
- **val** : the LD₅₀ values for acute toxicity (oral, rat) in mg substance per kg body weight

How do we make molecules machine-readable?

We need **molecular embeddings** that make our molecules readable for the machine. There are plenty of representations:



In this course, we will use **molecular fingerprints** as they are easy to generate with our cheminformatics toolbox RDKit but still encode plenty of information about the molecules for deriving our model.

To embed our molecules as fingerprints, we first need to understand what they exactly represent and how they are generated.

SMILES

SMILES = Simplified Molecular Input Line Entry System

SMILES is a text-based language to represent chemical structures as a linear string, while encoding:

- atoms
- bonds
- branching
- ring closures
- stereochemistry
- charges and isotopes

Despite being one-dimensional text, SMILES encode structural information about the molecule. You can derive all 2D information, however, no 3D information, since positions of the atoms are not encoded.

Atoms

Atom type / feature	SMILES notation	Example	Meaning
Organic subset atom	Element symbol	C , N , O	Common atoms, brackets not required
Aromatic atom	lowercase	c , n	Aromatic atom
Bracketed atom	[X]	[Fe]	Explicit atom with full specification
Charged atom	[X+], [X-]	[N+], [O-]	Formal charge
Explicit hydrogens	[XH], [XHn]	[NH], [NH4+]	Explicit hydrogen count
Isotope specification	[nX]	[13C]	Isotopically labeled atom
Chiral atom	@, @@	[C@H](O)N	Tetrahedral stereochemistry
Wildcard atom	*	*CC	Any atom (substructure queries)
Atom class (mapping)	:n	[C:1]	Atom mapping / reaction SMILES

| Dot-disconnect operator | . | [Na+].[Cl-] | For disconnections

Bonds

Bond type	SMILES symbol	Example	Meaning
Single bond	- (optional)	CC	C–C single bond
Double bond	=	C=C	C=C double bond
Triple bond	#	C#N	C≡N triple bond
Aromatic bond	:	c:c	Aromatic bond
Aromatic (implicit)	(none)	c1ccccc1	Aromatic system
E/Z stereochemical bond	/	F/C=C/F	Defines alkene geometry
E/Z stereochemical bond	\	F/C=C\F	Defines alkene geometry
Tetrahedral chirality (bond direction)	/ or \	N[C@H](O)C	Used with @ to define chirality

Examples

Molecule	SMILES
Water	<chem>O</chem>
Ethanol	<chem>CCO</chem>
2-Butanol	<chem>CC(C)CO</chem>
Acetic acid	<chem>CC(=O)O</chem>
L-Alanine	<chem>N[C@H](C)C(=O)O</chem>
Glucose	<chem>OC[C@H]1O[C@@H](O)[C@H](O)[C@@H](O)[C@H]1O</chem>

- hydrogen is encoded implicitly
- 2-butanol shows branching
- L-alanine explicitly encodes chirality with @
- glucose has many stereocenters, so SMILES becomes long quickly

Strength and limitations

Strength:

- encode the 2D molecular graph, incl. atoms, bonds, branching, ring closures
- bonds: single, double, triple
- aromaticity: captured via lowercase letters
- captures formal charges, valence states and resonance (implicity)
- stereochemistry: E/Z isomerism, tetrahedral chirality (@, @@)
- also isotopic labelling possible
- salts, mixtures can be express via disconnector dot
- reactions with atom-mapping

Limitations:

- sensitivity to string order
- many representations for one structure
- no conformational information, we need to apply other techniques for that (e.g. UFF using RDKit)
- no conformer distributions, no Boltzmann weighting
- no partial charges
- orbital information
- spin states
- excited states
- different tautomers, different SMILES, example N-methylmethanimine: C=NC and CN=C
- no multi-center bonds, metal coordination, no agostic interactions, no π -complexes
- limited for polymers: can only mimic parts of polymers, no molecular weight distribution, degree of polymerization
- dot-disconnected SMILES: does not distinguish between suspension, solution, salt, hydrogen-bonded complex etc.

SMILES in machine learning

Molecules are translated into "chemical language" using SMILES. This enables the use of Large Language Models (LLMs). However, with approximately **7000 datapoints** in our dataset, this is **not sufficient** to reliably train such models *from scratch*. The limitation is not SMILES themselves, but the fact that LLMs require large are "data-hungry": smaller LLMs use around 100 000 datapoints.

As a consequence, we need:

- a simpler model architecture compared to LLMs (explained later in more detail)
- **a different molecule embedding**

Molecular fingerprints

Definition

A molecular fingerprint is a vector that encodes the presence or absence of structural features in a molecule. Therefore, they describe molecules as fixed-length numerical embeddings that are readable for machines. These representations can be compared, searched and further processed, e.g. for machine learning.

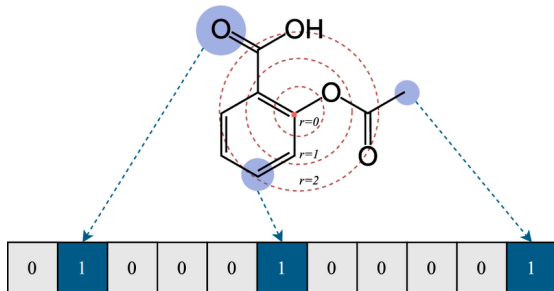
There are many different types of molecular fingerprints all with slightly different ways of the generation procedure: MACCS fingerprints, Morgan fingerprints, ECFP fingerprints

In a fingerprint, a molecule becomes a vector, usually filled with 0s and 1s indicating the absence (0) or presence (1) of predefined functional groups.

A single position in a fingerprint is called a **bit**:

- A fingerprint is a fixed-length vector, e.g.: 1024 bits, 2048 bits, 4096 bits
- Each bit stores:
 - 0 → feature absent
 - 1 → feature present

Molecular fingerprints are usually sized as powers of two (1024, 2048, 4096) because it makes hashing, memory alignment, and bit operations efficient. A bit **does not usually correspond to a specific chemical feature in the sense of a specific functional group**. Instead, a bit should be understood as a **bucket** that may represent one or more features after hashing.



from [Memetic Computing](<https://link.springer.com/article/10.1007/s12293-024-00414-6>), 'Nguyen-Vo et al., Molecular representations in bio-cheminformatics, Memetic Computing, 2024.'

Generation principle

The number of possible chemical features is extremely large. It is therefore impossible to allocate one bit per feature. The solution: **hashing**. Hashing is the step that connects **chemical features** to **bits** in a fixed-length fingerprint.

This section explains the process *conceptually*, without implementation details, and focuses on **what happens and why**.

Hashing maps an arbitrary chemical feature to a numerical value which is then folded into a fixed-length fingerprint vector

General workflow:

During fingerprint generation, the molecule is first decomposed into **features**, for example:

- an atom and its local neighborhood (ECFP)
- a path of bonded atoms
- a functional group pattern

Each feature is a **structured object**, not yet a number.

It contains information such as:

- atom types

- bond types
- aromaticity
- ring membership
- (optionally) stereochemistry

At this stage, the feature is still a *chemical description*. **A hash function turns these chemical descriptions into numbers.**

Conceptually, this means:

- the feature description is turned into a canonical representation
- this representation is passed to a **hash function**
- the hash function outputs a large integer

Key properties of the hash function:

- deterministic (same feature $\rightarrow$ same number)
- fast to compute
- distributes features uniformly over the number space

At this point, **similar features do not necessarily have similar numbers**: This means, hashing is not similarity-preserving.

The hash values are typically:

- very large (32-bit, 64-bit, or larger)
- sparse
- impractical to store directly for every feature

Moreover, the number of possible features is unbounded.

To obtain a **fixed-length fingerprint**, the hash values must be compressed.

The large hash value is folded into the fingerprint by computing a **bit index**. This step assigns the feature to **one of the available bit positions**.

It can happen that two different chemical features produce the **same hash value**. This happens **inside the hash function**, before folding into the fingerprint vector. By the **pigeonhole principle** hash-collisions are unavoidable because :

- Hash functions compress information
- Finite hash space
- Infinite number of possible features

Once the bit index is known:

- **binary fingerprints**
 - the bit is set to **1**
 - multiple features setting the same bit are indistinguishable
- **count fingerprints**
 - the value at that index is incremented
 - collisions accumulate counts

Importantly:

- the fingerprint stores **presence**, not identity
- the original feature cannot be recovered from the bit

Hashing **does not preserve**:

- chemical meaning of individual bits
- similarity between individual features
- uniqueness of features

The goal of fingerprints is not exact reconstruction, but:

- compactness
- speed
- statistical similarity

Hashing introduces two types of collisions:

1. **Hash collisions**

- different features → same hash value
- rare, internal to the hash function

2. **Bit collisions**: The folding procedure compresses many values into a few bins.

- different hash values → same bit index
- common, due to folding into fixed length

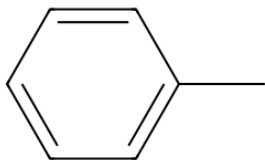
Both are accepted trade-offs for scalability.

Key take-homes

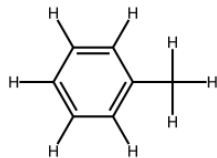
A fingerprint is a non-unique representation: You can always translate a molecule into a fingerprint but since information is compressed, you cannot directly translate a fingerprint into a molecular graph. Hashing converts rich chemical features into numerical identifiers, which are then folded into a fixed-size fingerprint by activating bits. This process compresses complex structural information into a fixed-length vector, enabling scalable similarity search and machine-learning applications.

Now, let's first have a look at a single molecule (toluene) to sketch the basic principles on how RDKit works and how to produce fingerprints. Then, add more complexity while moving to our dataset.

Molecule without hydrogen:



Molecule with hydrogen:



Now we use our Toluene mol-object from RDKit that we generated using the SMILES and generate a binary, circular fingerprint from that:

[illegible]

Data cleaning

Before we apply this fingerprint generation to our whole dataset, we first need to make sure that the dataset is cleaned, which means:

- no multiple entries

- each structure has a value assigned and vice-versa

Checking for multiple entries

We already learned that SMILES is not a unique representation of a molecule: One molecule can be encoded by different SMILES, such as ethanol: `CCO` or `θCC` or `C(C)θ`. To check for duplets, we cannot just compare the SMILES. However, there is one solution to that problem:

Canonicalization.

Canonical SMILES are a unique SMILES string assigned to a molecule by RDKit (or other cheminformatics tools), so that:

- Each molecule → exactly one canonical SMILES
- Equivalent molecules (isomers aside) → same canonical SMILES
- Ensures reproducibility and consistent hashing/fingerprints

1. RDKit convert the SMILES into a molecular graph: Atoms become nodes, bonds become edges, hydrogens are treated implicitly
2. Unique atom rankings: RDKit assigns a unique integer to each atom (based on graph topology, element type, connectivity, and optionally stereochemistry). This is called atom invariants.
3. Determine canonical traversal order: RDKit uses the atom rankings to decide a unique traversal order of the molecular graph. This traversal is deterministic — same molecule → same order.
4. Write SMILES in traversal order. RDKit outputs a SMILES string following that traversal order. Ring closures and branches are numbered deterministically.
5. Optional stereochemistry encoding: If `isomericSmiles=True` in `Chem.MolToSmiles()`, stereochemistry (`@`, `/`, `\`) is included.

Let's canonicalize our SMILES in the dataset!

```
[19:31:09] Explicit valence for atom # 1 Si, 4, is greater than permitted
[19:31:09] WARNING: not removing hydrogen atom with neighbor that has non-tetrahedral stereochemistry
```

First, let's check for missing values. Is there any structure without value or canonical smiles?

```
Unnamed: 0      0
SMILES         0
val            0
canonical_smiles 1
dtype: int64
```

We can see that:

- there is an experimental value assigned to each structure
- but there is 1 structure where the canonical SMILES is empty. The canonicalization failed. This can happen if the SMILES in the original dataset already contain errors. Let's inspect the molecule and delete this datapoint.

```
      Unnamed: 0      SMILES      val canonical_smiles
7194      7194  [C-]#[SiH+]  2001.0          None
```

We now have two options:

- try to fix canonicalization
- delete the datapoint

Our goal is to implement a model that determines **the acute toxicity of solvents**. Solvent molecules usually have a very different structure. Therefore: Let's simply delete this datapoint.

Check again: Number of missing entries:

```
Unnamed: 0      0
SMILES         0
val            0
canonical_smiles 0
dtype: int64
```

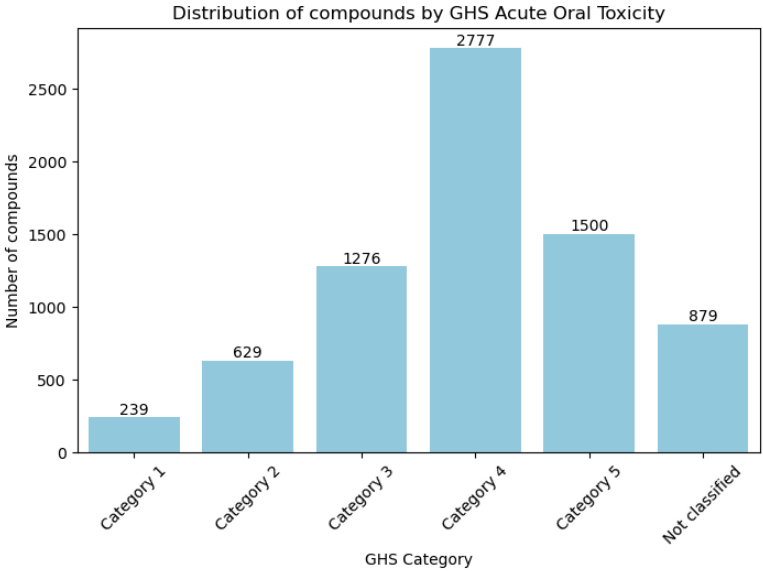
Looks good now! Now we need to check whether there are multiple datapoints of the same chemical. If so we could merge multiple entries into one by taking the average.

Number of entries in the original dataset:
7300
Number of entries in the dataset with erased entries:
7300

Same number as before! That means, our dataset did not contain duplets, nice!

Data analysis

After the initial analysis, let's inspect our data a bit deeper. Since the model learns from data, the model can only describe well what it has seen. What are the limits in terms of structural and value ranges? Let's have a look at the LD₅₀ values and make a histogram plot of how many extremely toxic and less toxic substances we have.



That was easy! We see that most chemicals are Category 4 and only few are Category 1. We need to take this into account when training the model! More explanations on that later when we implement the model.

Now we need to also analyze the **structural data**. Here is how we can go from SMILES to visualizing our chemical space.

1. Convert canonical SMILES into fingerprints
2. Dimensionality reduction
3. Plotting the reduced structural information vs toxicity values

Can we already relate some structural features to toxicity?

Let's generate the fingerprints:

Unnamed: 0	SMILES	val	\
0	0	Br/C=C\Br	62.0
1	1	BrC(Br)(Br)c1ccc2ccccc2n1	6769.0
2	2	BrC(Br)Br	933.0
3	3	BrC(Br)C(Br)Br	1200.0
4	4	BrC/C=C/CBr	62.0
...	...	...	...
7296	7296	c1coc(0CC2C02)c1	1100.0
7297	7297	c1nc(NC2CC2)c2ncn(C3CC3)c2n1	130.0
7298	7298	c1nc(NC2CC2)c2ncn(CC3CC3)c2n1	160.0
7299	7299	c1nc[nH]n1	1648.0
7300	7300	c1nnc(NCnc2nncs2)s1	260.0

canonical_smiles	GHS_category	\
Br/C=C\Br	Category 3	
BrC(Br)(Br)c1ccc2ccccc2n1	Not classified	
BrC(Br)Br	Category 4	
BrC(Br)C(Br)Br	Category 4	
BrC/C=C/CBr	Category 3	
...	...	...
c1coc(0CC2C02)c1	Category 4	
c1nc(NC2CC2)c2ncn(C3CC3)c2n1	Category 3	
c1nc(NC2CC2)c2ncn(CC3CC3)c2n1	Category 3	
c1nc[nH]n1	Category 4	
c1nnc(NCnc2nncs2)s1	Category 3	

	fingerprint
0	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
1	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
2	[0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
3	[0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
4	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
...	...
7296	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
7297	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, ...
7298	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, ...
7299	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
7300	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...

[7300 rows x 6 columns]

[19:31:12] WARNING: not removing hydrogen atom with neighbor that has non-tetrahedral stereochemistry

Our structural information is now stored in the fingerprints, each having 2048 bits. However, we cannot plot 2048 dimensions. Let's reduce this dimension to 2, so we can visualize the structures them in a 2D scatter plot.

Techniques for dimensionality reduction are:

- Principal Component Analysis (PCA): Find the directions (principal components) that capture the most variance in the data. Only captures linear relationships, may miss complex chemical patterns.
- t-SNE (t-distributed Stochastic Neighbor Embedding): Preserves local structure — molecules that are similar in high dimensions stay close in 2D. (non-linear method)
- UMAP (Uniform Manifold Approximation and Projection): Non-linear, preserves both local and global structure better than t-SNE in many cases. (non-linear method)

t-SNE

As we will use t-SNE, we will look a bit deeper in how it works. t-SNE stands for t-distributed Stochastic Neighbor Embedding and is a dimensionality-reduction method mainly used for visualization.

- maps high-dimensional data to 2D or 3D while preserving local neighborhood structure.
- points that are close together in the original space tend to stay close in the embedding.
- distances between far-apart clusters are not meaningful and should not be over-interpreted.
- it is not a predictive model and cannot easily embed new points without rerunning.

t-SNE is best used for exploration and visualization, not for quantitative analysis.

How is it different from PCA and UMAP?

PCA: Linear method, preserves maximum variance in the data

UMAP: Nonlinear method, preserves local neighborhoods and some global structure, much faster than t-SNE, handles large datasets well

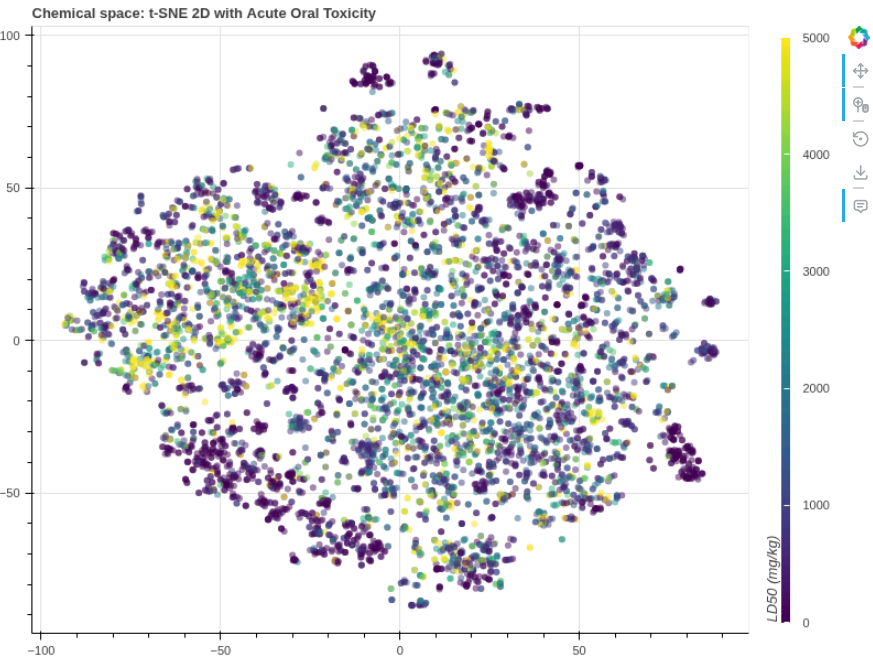
Now, we will learn how to reduce the high-dimensional fingerprint into 2D so we can plot the structural space.

```
c:\Users\koenigmattern\AppData\Local\miniforge3\envs\teaching\Lib\site-packages\sklearn\metrics\pairwise.py:2459: DataConversionWarning: Data was converted to boolean for metric jaccard
warnings.warn(msg, DataConversionWarning)
```

Great! We saved both t-SNE-derived dimensions for each chemical in the dataframe. Now we want to plot the results: Plot the structures in 2D space and give each dot - corresponding to a chemical - a color representing the LD50.

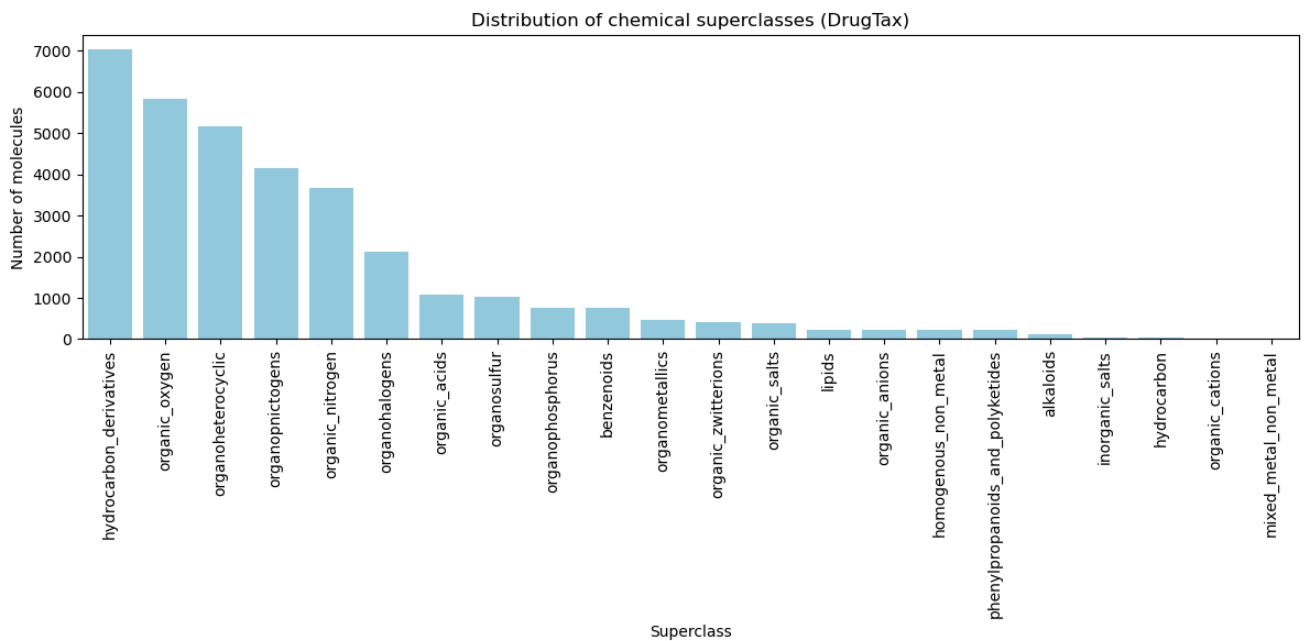
```
[19:33:09] WARNING: not removing hydrogen atom with neighbor that has non-tetrahedral stereochemistry
```

Now this graphic should pop up in your browser:



In case you encounter problems in colab, just download it from the [repo](#) and view in your browser. Enjoy!

Now let's use another way to visualize the structural space. Let's classify each molecule (e.g. hydrocarbon, lipids,) and the count each occurrence. Note: Since one molecule can be assigned to one or more categories, there will be more counts than molecules in the dataset.



We see that while for some classes, there are plenty datapoints, there are only few for e.g. metal-containing chemicals and ionic species. Our model will have limited predictive power for those chemicals with few data available!

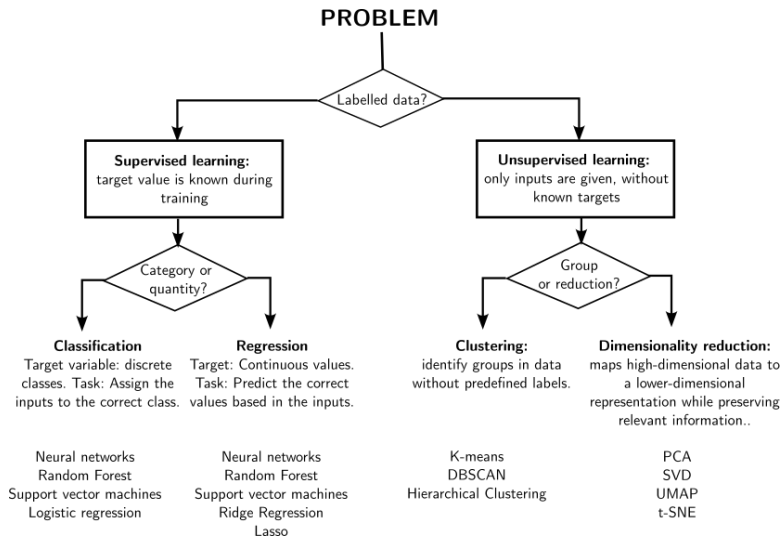
Recap

- We have made our molecules "machine-readable" by converting them to fingerprints
- We have collected, cleaned and analyzed our dataset

Next step Model implementation

Model implementation

Selecting the right algorithm



The algorithm to be chosen also largely depends on the size and the quality of the dataset. As a rule of thumb, choose these algorithms as a first try:

- 50 - 1000 datapoints: Linear models, Random Forest, Support Vector Machines, Gradient Boosting, Gaussian Process Models
- 1000 - 10000 datapoints: Random Forest, Gradient Boosting, Graph Neural Networks, Deep Neural Networks
- 10000 - 100000 datapoints: Graph Neural Networks, Deep Neural Networks, Transfer Learning
- > 100000: Large Language Models, Transformers

Considerations for our model

- Our dataset has input and output given: Supervised learning
- Goal: predict continuous values of LD₅₀ → Regression task
- Dataset size: Our dataset is too small for large language models but would be suitable for Random Forests, Gradient Boosting or Graph Neural Networks.

Graph Neural Networks are very powerful (as you will see later) but a bit more complex to implement. Due to time reasons, we choose the **Random Forest Algorithm** for our problem.

Regression: Random Forest Algorithm

Random Forest (RF) is an ensemble learning algorithm that combines many simple decision trees to make a robust prediction. Instead of relying on a single decision tree, a random forest builds many different trees and combines their predictions. What is a decision tree? A decision tree is a model that makes predictions by asking a sequence of simple yes/no questions about the input features.

- Start with all data at the root
- Split the data based on one feature (e.g. "Is this fingerprint bit = 1?")
- Repeat splitting until the data is sufficiently pure
- Make a prediction at the leaf nodes

In the RF algorithm, multiple trees are combined, since training only on one tree is prone to overfitting (we'll see later what this exactly means)

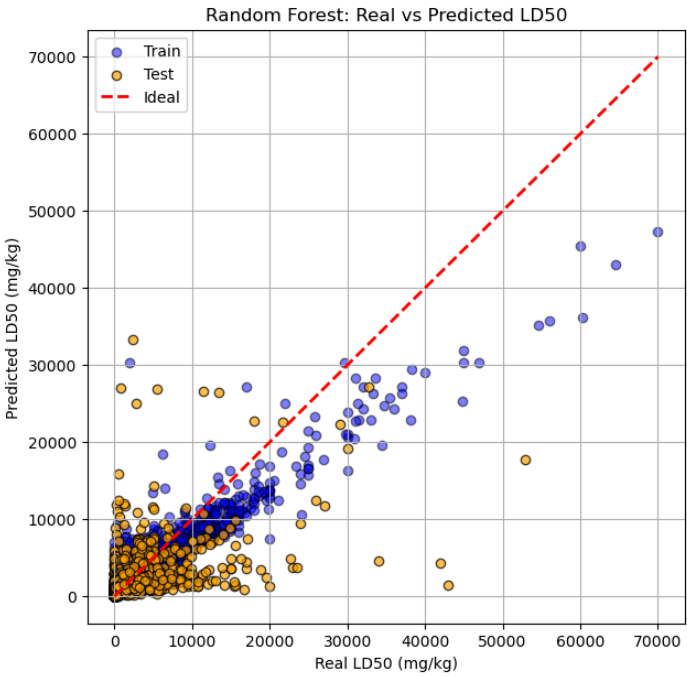
The RF algorithm follows these steps:

- 1. Bootstrap sampling: Each tree is trained on a random subset of the data (sampling with replacement).
- 2. Random feature selection: At each split in a tree, only a random subset of features is considered.
- 3. Grow many trees: Each tree learns slightly different decision rules.
- 4. Aggregate predictions: Regression: average over all trees, Classification: majority vote

More detailed information: Breiman, “Random Forests”, Machine Learning, 45(1), 5-32, 2001. , P. Geurts, D. Ernst., and L. Wehenkel, “Extremely randomized trees”, Machine Learning, 63(1), 3-42, 2006.

But before we can train our RF model, we need to split our dataset into training and testing. The training set is used for building the actual model, in general contains about 70 - 80 % of the data. The test set is used to evaluate the model performance and contains about 30 - 20 % of the data. Let's split the data and train the model!

Train metrics: RMSE=1616.29, MAE=729.17, R2=0.862
Test metrics: RMSE=3786.06, MAE=1886.86, R2=0.167



Regression Evaluation Metrics

Let y_i be the true values, $\hat{y}_i$ the predicted values, and n the number of samples.

Mean Absolute Error (MAE)

Definition:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Explanation:

- Measures the average absolute difference between predictions and true values.
- Has the same units as the target variable.
- Penalizes all errors linearly and is relatively robust to outliers.

Root Mean Squared Error (RMSE)

Definition:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Explanation:

- Squares errors before averaging, then takes the square root.
- Penalizes large errors more strongly than MAE.
- Has the same units as the target variable.

Coefficient of Determination R^2

Definition:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

where $\bar{y}$ is the mean of the true values.

Explanation:

- Measures the fraction of variance in the data explained by the model.
- Compares the model to a baseline that predicts the mean.
- Dimensionless quantity.

Interpretation:

- $R^2 = 1$: perfect prediction
- $R^2 = 0$: no improvement over the mean
- $R^2 < 0$: worse than predicting the mean

We see that the model does not perform well in its current form: The MAE is high (spans multiple toxicity categories) and the R^2 is low.

In its current form, the model needs to handle values ranging from close to 0 up to 50000.

On the raw scale:

- values are typically heteroscedastic (larger variance at higher values)
- small LD₅₀ differences at low values (e.g. 5 → 10 mg kg⁻¹) are toxicologically huge
- large absolute differences at high values (10 000 → 20 000 mg kg⁻¹) are often less meaningful

Different scales are hard to handle for ML algorithms!

In our case, it makes sense to transform our values to a logarithmic scale because most regression models implicitly assume:

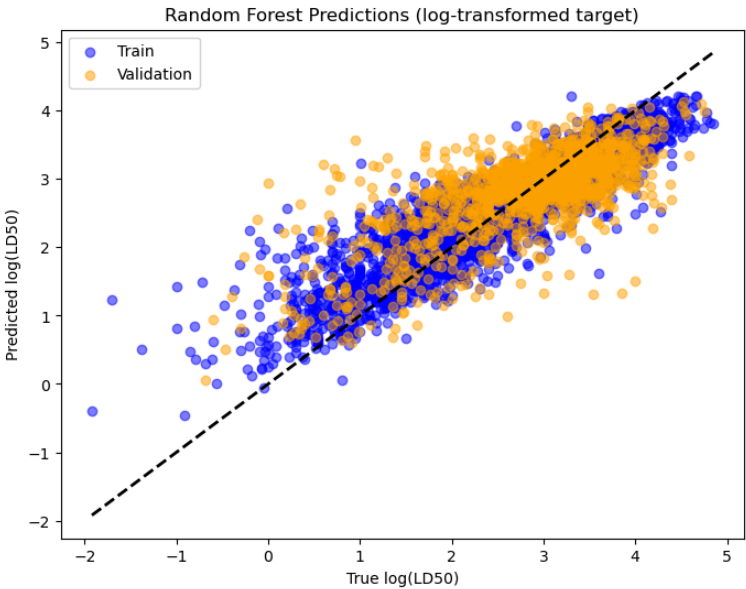
- reduces heteroscedasticity
- errors of similar scale across the target range
- roughly symmetric distributions
- makes the distribution more symmetric

This leads to:

- more stable training
- better generalization
- more meaningful uncertainty estimates

A log transform compresses the scale so that relative differences matter, not absolute ones.

```
{'MAE Train': 0.2685447760296197, 'RMSE Train': np.float64(0.3774024205197653), 'R2 Train': 0.8191618218962406, 'MAE Val': 0.4577278369624473, 'RMSE Val': np.float64(0.6141559290277971), 'R2 Val': 0.5149330043083351}
```



The scatter plot now shows:

- lower accuracy at low LD50, higher accuracy at high LD50 --> difference to raw scale

Looking at the metrics:

- MAE is about 0.5 in $\log_{10}(\text{LD}_{50})$. This corresponds to a multiplicative error of about a factor of 3 ($10^{0.5} = 3.16$). The model predicts LD_{50} values within a factor of ~3 on average.
- R^2 is still relatively low. This could be caused by:
 - missing descriptors (we could add more descriptors using RDKit easily)

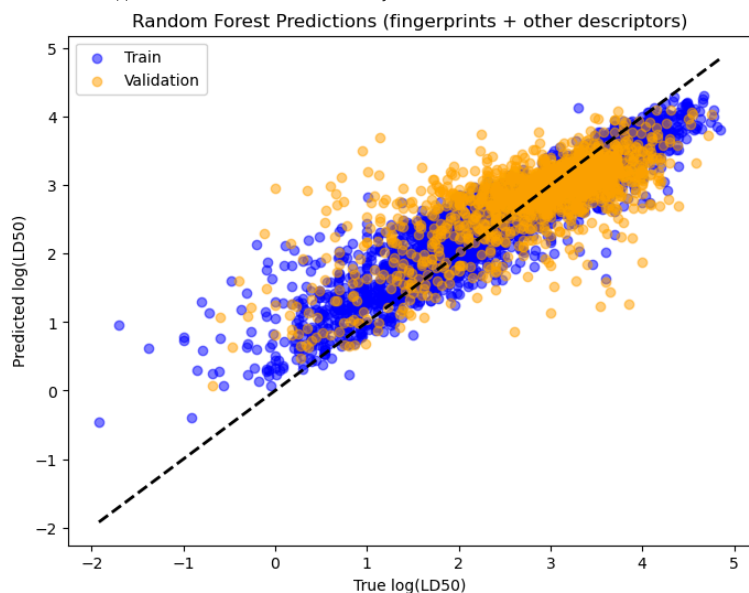
- intrinsic noise in LD₅₀ data (we cannot change that)
- limited sample size (we cannot change that either)
- that the structure-toxicity relationship is weak (if so, we can't change that)

Take-homes: On the raw scale, the model is dominated by large LD₅₀ values → low LD₅₀ compounds look badly predicted. On the log scale, the model optimizes relative (multiplicative) error, so low LD₅₀ values are no longer overwhelmed and high LD₅₀ values can tolerate larger absolute errors. On the log scale, accuracy becomes more uniform in relative terms, not necessarily higher everywhere.

Note: Take into account that the error metrics are given in log-unit now as well, so they cannot be directly compared.

Next, we could try adding more features in addition to the fingerprints.

```
[19:39:38] WARNING: not removing hydrogen atom with neighbor that has non-tetrahedral stereochemistry
{'MSE Train': 0.11481331843599836, 'MAE Train': 0.2377532925219125, 'RMSE Train': np.float64(0.33884114041243335), 'R2 Train': 0.8542283633032236, 'MSE Val': 0.36579905724064254, 'MAE Val': 0.44301808733029485, 'RMSE Val': np.float64(0.6048132416214468), 'R2 Val': 0.5295786650003266}
```



Unfortunately, adding these descriptors does not improve the accuracy.

Do we really need absolute toxicity values or is it sufficient to discriminate between toxic and non-toxic molecules?

Classification

Let's move away from a regression model to a classification model.

What is classification? Classification is a type of supervised machine learning problem where the goal is to assign each input (sample) to one of a set of discrete categories (classes) based on its features.

Input: Features describing the sample (numeric, categorical, or mixed).

Output: A discrete label or class (e.g., 0/1, Red/Green/Blue, Toxic/Non-toxic).

Goal: Learn a mapping from inputs → classes that generalizes to new, unseen samples.

Classification Type	Definition	Example
Binary Classification	Each sample belongs to one of two possible classes .	Predict whether a solvent is toxic or non-toxic .
Multiclass Classification	Each sample belongs to exactly one of three or more classes .	Predict the hazard category of a chemical: Low / Medium / High.
Multilabel Classification	Each sample can belong to multiple classes simultaneously .	Predict all applicable hazards of a solvent: Toxic, Flammable, Corrosive.

Considerations for our problem: For our case it's sufficient to discriminate between toxic and non-toxic based on the molecular fingerprint. As a next step, we need to clearly define, what we understand under "toxic" and "non-toxic":

- we only consider acute toxicity
- toxic: every molecule from Category 1 - 4 is considered as toxic
- non-toxic: molecules rated as Category 5 or with even higher LD₅₀s, are rated as non-toxic

We can reformulate our regression problem as a classification problem

Train classification report:					
	precision	recall	f1-score	support	
	0	0.850	0.921	0.884	1903
	1	0.960	0.922	0.941	3937
accuracy			0.921		5840
macro avg	0.905	0.921	0.912		5840
weighted avg	0.924	0.921	0.922		5840

Validation classification		report:		
	precision	recall	f1-score	support
0	0.608	0.603	0.605	476
1	0.809	0.812	0.810	984
accuracy			0.744	1460
macro avg			0.708	1460
weighted avg			0.743	1460

Validation confusion matrix:
[[287 189]
[185 799]]

Evaluation metrics for binary classification

Confusion Matrix (Binary Classification)**

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

All classification metrics are derived from these four quantities.

Accuracy:

Definition:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Explanation

- Fraction of correctly classified samples.
- Easy to interpret, but often misleading.

Limitation

- Not suitable for imbalanced datasets.

Precision (Positive Predictive Value)

Definition:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Explanation:

- Of all samples predicted as positive, how many are truly positive?
- Penalizes false positives.

Use when:

- False positives are costly.

Recall (Sensitivity, True Positive Rate)

Definition:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Explanation:

- Of all actual positive samples, how many were correctly identified?
- Penalizes false negatives.

Use when:

- Missing positive samples is costly.

F1-score

Definition:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Explanation

- Harmonic mean of precision and recall.
- Balances false positives and false negatives.
- Useful for imbalanced datasets.

With this knowledge, let's evaluate our classification results!

Training Performance

The model shows strong performance on the training data, with an overall accuracy of **0.921**.

- **Class 0 (non-toxic):**
 - Precision = 0.85
 - Recall = 0.921
 - F1-score = 0.884The model correctly identifies most non-toxic compounds, with a small number of false positives.
- **Class 1 (toxic):**
 - Precision = 0.96
 - Recall = 0.922
 - F1-score = 0.941The toxic class is predicted with very high precision and recall, indicating that toxic compounds are both rarely missed and rarely misclassified.

Overall, the training results indicate a well-performing model with slightly better performance for the toxic class than for the non-toxic class.

Validation Performance

On the validation set, performance drops noticeably, with an overall accuracy of **0.744**, indicating reduced generalization.

- **Class 0 (non-toxic):**
 - Precision = 0.608
 - Recall = 0.603
 - F1-score = 0.605The model struggles to correctly identify non-toxic compounds, with a substantial number of false positives and false negatives.
- **Class 1 (toxic):**
 - Precision = 0.809
 - Recall = 0.812
 - F1-score = 0.810Performance for the toxic class remains relatively strong, showing that the model is more reliable at detecting toxic compounds than non-toxic ones.

Confusion Matrix Analysis

Confusion matrix (validation set):

$$\begin{pmatrix} 287 & 189 \\ 185 & 799 \end{pmatrix}$$

- **True non-toxic (TN):** 287
- **False toxic (FP):** 189
- **False non-toxic (FN):** 185
- **True toxic (TP):** 799

The confusion matrix confirms that:

- Toxic compounds are identified reasonably well.
- Some non-toxic compounds are incorrectly classified as toxic.
- The model exhibits a **bias toward predicting the toxic class**, which explains the higher recall and precision for toxic compounds compared to non-toxic ones.

From a safety perspective, prioritizing recall for the toxic class may be acceptable, but at the cost of some false alarms.

Using a graph neural network for lignin solubility prediction

Graph Neural Networks (GNNs): Definition

Graph Neural Networks (GNNs) are a class of machine learning models designed to operate on **graph-structured data**. Unlike traditional neural networks, which assume data lies on regular grids (e.g. vectors, images, sequences), GNNs explicitly model **entities and their relationships**.

Graphs are defined as:

- **Nodes (vertices)**: entities (e.g. atoms, users, intersections)
- **Edges**: relationships between entities (e.g. chemical bonds, friendships, roads)
- **Optional features** on nodes, edges, or the entire graph

Node, Edge, and Graph Features

GNNs can incorporate:

- **Node features**: e.g. atom type (C, N, O S, etc.), charge, hybridization
- **Edge features**: bond type (single, double triple), aromaticity, ring
- **Global (graph-level) features**: temperature, pressure, molecular descriptors

Why Graph Neural Networks?

Many real-world systems are naturally graphs:

- **Chemistry & materials**: atoms connected by bonds
- **Biology**: proteins, gene regulatory networks
- **Social networks**: people and their interactions
- **Knowledge graphs**: entities and relations
- **Engineering systems**: process flows, power grids

Principle: Message Passing

Most GNNs are based on **message passing**. Each node updates its representation by:

1. **Receiving messages** from its neighboring nodes
2. **Aggregating** those messages (using an aggregation function, e.g. sum, mean, max, or attention-based)
3. **Updating** its own state using a neural network (using an update function that combines the node's original features with the aggregated messages)

This process is repeated over multiple layers, allowing information to propagate through the graph.

Readout / Pooling

To make **graph-level predictions**, node representations must be aggregated into a single vector that contains information about structural features of the graph, very similar to a GNN-derived molecular fingerprint.

Property prediction

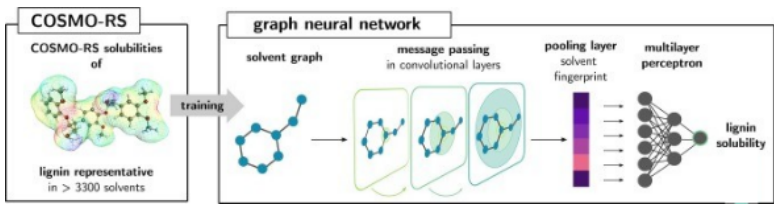
The vector obtained from pooling finally serves as an input to another machine learning model (such as a multilayer perceptron) to predict the target property.

Training a graph neural network for lignin solubility predictions

Dataset

Since experimental values are scarce, we used the DFT-based method COSMO-RS to predict lignin solubilities. We represented lignin by a simplified model structure (trimer of G-units) and predicted its solubilities in ca. 3300 solvents.

GNN implementation



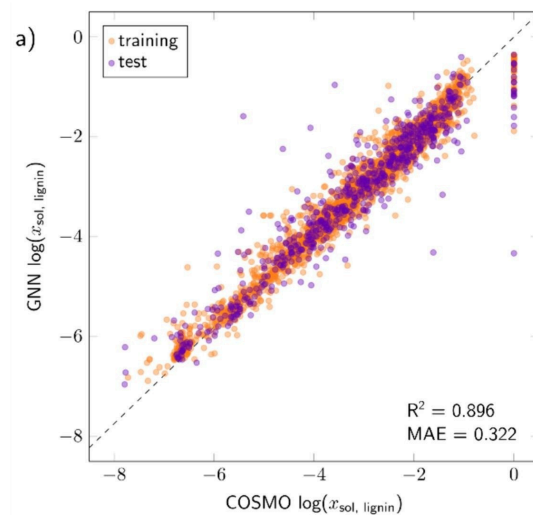
adapted from [Chemical Engineering Journal](<https://www.sciencedirect.com/science/article/pii/S2211339819300619>), "König-Mattern et al., Machine learning-supported solvent design for lignin-first biorefineries and lignin upgrading, CEJ, 2024."

Bond and atom features

Atom or bond	Feature	Description	Dimension
Atom features	Atom type	C, O, N, Cl, F, S, Si, Br, P, Se, I, B, As, Ge, Al	15
	Ring	Is it in ring?	1
	Aromatic	Is it aromatic?	1
	Hybridisation	sp, sp2, sp3, sp3d	4
	Bonds	Number of bonds attached (0, 1, 2, 3, 4)	5
	Charge	Formal charge (0, 1, -1, 3)	4
	H's attached	Number of bonded H's (0, 1, 2, 3)	4
Bond features	Bond type	single, double, triple, aromatic	4
	Conjugated	Is it conjugated?	1

| | Ring | Is it in ring?

Results



adapted from [Chemical Engineering Journal][<https://www.sciencedirect.com/science/article/pii/S2211339819300619>], König-Mattern et al., Machine learning-supported solvent design for lignin-first biorefineries and lignin upgrading, CEJ, 2024.

Code availability

If you want to deep-dive into the code, you can find it (here)[https://github.com/koenigmattern/PSEvolve_lignin_solvents]. Have fun and feel free to contact me in case you have any questions!

Solvent selection

Goal: We want to identify solvents that

1. are non-toxic (as predicted by our binary classification model for acute toxicity)
2. have maximum lignin solubility as predicted by the GNN
3. low price

State of the art tools for solvent selection

Chem21 solvent selection guide:

Family	Solvent	BP (°C)	FP (°C)	Worst H3xx ^a	H4xx	Safety score	Health score	Env. score	Ranking by default	Ranking after discussion ^b
Water	Water	100	na	None	None	1	1	1	Recommended	Recommended
	MeOH	65	11	H301	None	4	7	5	Problematic	Recommended
	EtOH	78	13	H319	None	4	3	3	Recommended	Recommended
	i-PrOH	82	12	H319	None	4	3	3	Recommended	Recommended
	n-BuOH	118	29	H318	None	3	4	3	Recommended	Recommended
	t-BuOH ^c	82	11	H319	None	4	3	3	Recommended	Recommended
Alcohols	Benzyl alcohol	206	101	H302	None	1	2	7	Problematic	Problematic
	Ethylene glycol	198	116	H302	None	1	2	5	Recommended	Recommended
	Acetone	56	-18	H319	None	5	3	5	Problematic	Recommended
	MEK	80	-6	H319	None	5	3	3	Recommended	Recommended
	MIBK	117	13	H319	None	4	2	3	Recommended	Recommended
	Cyclohexanone	156	43	H332	None	3	2	5	Recommended	Problematic
Ketones	Methyl acetate	57	-10	H302	None	5	3	5	Problematic	Problematic
	Ethyl acetate	77	-4	H319	None	5	3	3	Recommended	Recommended
	i-PrOAc	89	2	H319	None	4	2	3	Recommended	Recommended
	n-BuOAc	126	22	H336	None	4	2	3	Recommended	Recommended
	Diethyl ether	34	-45	H302	None	10	3	7	Hazardous	III
	Diisopropyl ether	69	-28	H336	None	9	3	5	Hazardous	Hazardous
Esters	MTBE	55	-28	H315	None	8	3	5	Hazardous	Hazardous
	THF	66	-14	H351	None	6	7	5	Problematic	Problematic
	Me-THF	80	-11	H318	None	6	5	3	Problematic	Problematic
	1,4-Dioxane	101	12	H351	None	7	6	3	Problematic	Hazardous
	Anisole	154	52	None	None	4	1	5	Problematic	Recommended
	DME	85	-6	H360	None	7	10	3	Hazardous	Hazardous
Hydrocarbons	Pentane	36	-40	H304	H411	8	3	7	Hazardous	Hazardous
	Hexane	69	-22	H361	H411	8	7	7	Hazardous	Hazardous
	Heptane	98	-4	H304	H410	6	2	7	Problematic	Problematic
	Cyclohexane	81	-17	H304	H410	6	3	7	Problematic	Problematic
	Me-cyclohexane	101	-4	H304	H411	6	2	7	Problematic	Problematic
	Benzene	80	-11	H350	None	6	10	3	Hazardous	III
Halogenated	Toluene	111	4	H351	None	5	6	3	Problematic	Problematic
	Xylenes	140	27	H312	None	4	2	5	Problematic	Problematic
	DCM	40	na	H351	None	1	7	7	Hazardous	Hazardous
	Chloroform	61	na	H351	None	2	7	5	Problematic	III
	CCl ₄	77	na	H351	H420	2	7	10	Hazardous	III
	DCE	84	13	H350	None	4	10	3	Hazardous	III
Aprotic polar	Chlorobenzene	132	29	H332	H411	3	2	7	Problematic	Problematic
	Acetonitrile	82	2	H319	None	4	3	3	Recommended	Problematic
	DMF	153	58	H360	None	3	9	5	Hazardous	Hazardous
	DMAc	166	70	H360	None	1	9	5	Hazardous	Hazardous
	NMP	202	96	H360	None	1	9	7	Hazardous	Hazardous
	DMPU	246	121	H361	None	1	6	7	Problematic	Problematic
Miscellaneous	DMSO ^c	189	95	None	None	1	1	5	Recommended	Problematic
	Sulfolane ^c	287	177	H360	None	1	9	7	Hazardous	Hazardous
	HMPA	200	144	H350	None	1	9	7	Hazardous	III
	Nitromethane	101	35	H302	None	10	2	3	Hazardous	III
	Methoxy-ethanol	125	42	H360	None	3	9	3	Hazardous	Hazardous
	Carbon disulfide	46	-30	H361	H412	9	7	7	Hazardous	III
Acids	Formic acid	101	49	H314	None	3	7	3	Problematic	Problematic
	Acetic acid	118	39	H314	None	3	7	3	Problematic	Problematic
	Ac ₂ O	139	49	H314	None	3	7	3	Problematic	Problematic
Amines	Pyridine	115	23	H302	None	4	2	3	Recommended	Hazardous
	TEA	89	-6	H314	None	6	7	3	Problematic	Hazardous

from [Green Chemistry](https://pubs.rsc.org/en/content/articlelanding/2016/gc/c5gc01008)), ^aPrat et al., CHEM21 selection guide of classical- and less classical-solvents, Green Chemistry, 2016.

ACS solvent selection tool:

You can find the solvent selection tool from the American Society of Chemistry [here](#)

Computational solvent screening

For a solvent screening, we need to evaluate a small but well-defined part of the chemical space and predict the target properties using our derived models.

To identify the optimal solvents, we need to follow these steps:

1. Define the solvent space: Collect a list of solvents
2. Predict the target properties: Apply the models to predict toxicity and lignin solubility
3. Rank the solvents: Define metrics for the ranking
4. Discuss the solutions

Therefore, let's first focus on defining our search space. For our example, we take the solvent list published [here](#) containing 500 commonly used solvents, including their SMILES and price.

	Solvent	Smiles	Cost
0	Methyl Acetate	CC(OC)=O	0.020
1	2,3-Dihydrofuran	O1CCC=C1	0.368
2	Methyl-t-Butyl Ether	CC(C)(C)OC	0.020
3	2-Pentyne	CC#CCC	3.335
4	Ethyl Methyl Sulfide	CSCC	0.761
...	...	...	...
494	Nonyl Mercaptan	CCCCCCCCCS	1.250
495	Tert-Butyl Mercaptan	S[C@](C)(C)C	0.186
496	Tert-Butylbenzene	CC(C)(C1=CC=CC=C1)C	0.062
497	Methyl-t-Butyl Ether	CC(C)(C)OC	0.040
498	Ethylene Dichloride	C(CCl)Cl	0.046

[499 rows x 3 columns]

Property prediction

We still need to predict the lignin solubility (given in log-units) and the toxicity (binary: 0 - nontoxic, 1 - toxic)

```
c:\Users\koenigsmattn\AppData\Local\miniforge3\envs\teaching\python.exe
c:\Users\koenigsmattn\OneDrive - Center for the Transformation of Chemistry\Dokumente\Code\sustainable_chemical_systems_course\code\Trained_GNN_LC
c:\Users\koenigsmattn\OneDrive - Center for the Transformation of Chemistry\Dokumente\Code\sustainable_chemical_systems_course\code\Trained_GNN_LC\GNN_LC.py:101: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  model.load_state_dict(torch.load(path_model_info))
c:\Users\koenigsmattn\OneDrive - Center for the Transformation of Chemistry\Dokumente\Code\sustainable_chemical_systems_course\code\Trained_GNN_LC\GNN_LC.py:101: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  model.load_state_dict(torch.load(path_model_info))
c:\Users\koenigsmattn\OneDrive - Center for the Transformation of Chemistry\Dokumente\Code\sustainable_chemical_systems_course\code\Trained_GNN_LC\GNN_LC.py:101: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  model.load_state_dict(torch.load(path_model_info))
c:\Users\koenigsmattn\OneDrive - Center for the Transformation of Chemistry\Dokumente\Code\sustainable_chemical_systems_course\code\Trained_GNN_LC\GNN_LC.py:101: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  model.load_state_dict(torch.load(path_model_info))
```

	Solvent	Smiles	Cost	canonical_smiles	\
0	Methyl Acetate	CC(OC)=O	0.020	COC(C)=O	
1	2,3-Dihydrofuran	O1CCC=C1	0.368	C1=C0CC1	
2	Methyl-t-Butyl Ether	CC(C)(C)OC	0.020	COC(C)(C)C	
3	2-Pentyne	CC#CCC	3.335	CC#CCC	
4	Ethyl Methyl Sulfide	CSCC	0.761	CCSC	
...	...	...	...	...	
494	Nonyl Mercaptan	CCCCCCCCCS	1.250	CCCCCCCCCS	
495	Tert-Butyl Mercaptan	S[C@](C)(C)C	0.186	CC(C)(C)S	
496	Tert-Butylbenzene	CC(C)(C1=CC=CC=C1)C	0.062	CC(C)(C)c1ccccc1	
497	Methyl-t-Butyl Ether	CC(C)(C)OC	0.040	COC(C)(C)C	
498	Ethylene Dichloride	C(CCl)Cl	0.046	ClCCCl	

	GNN_prediction_lignin
0	-1.675693
1	-2.594162
2	-4.766807
3	-3.590062
4	-3.748214
...	...
494	-4.422548
495	-3.563515
496	-4.729118
497	-4.766807
498	-2.527569

[499 rows x 5 columns]

c:\Users\koenigmattern\OneDrive - Center for the Transformation of Chemistry\Dokumente\Code\sustainable_chemical_systems_course\code\Trained_GNN_LC\GNN_LC.py:101: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```
model.load_state_dict(torch.load(path_model_info))
```

	Solvent	Smiles	Cost	canonical_smiles	\
0	Methyl Acetate	CC(OC)=O	0.020	COC(C)=O	
1	2,3-Dihydrofuran	O1CCC=C1	0.368	C1=C0CC1	
2	Methyl-t-Butyl Ether	CC(C)(C)OC	0.020	COC(C)(C)C	
3	2-Pentyne	CC#CCC	3.335	CC#CCC	
4	Ethyl Methyl Sulfide	CSCC	0.761	CCSC	
...	...	...	...	...	
494	Nonyl Mercaptan	CCCCCCCCCS	1.250	CCCCCCCCCS	
495	Tert-Butyl Mercaptan	S[C@](C)(C)C	0.186	CC(C)(C)S	
496	Tert-Butylbenzene	CC(C)(C1=CC=CC=C1)C	0.062	CC(C)(C)c1ccccc1	
497	Methyl-t-Butyl Ether	CC(C)(C)OC	0.040	COC(C)(C)C	
498	Ethylene Dichloride	C(CCl)Cl	0.046	ClCCCl	

	GNN_prediction_lignin	acute_toxicity
0	-1.675693	0
1	-2.594162	1
2	-4.766807	0
3	-3.590062	1
4	-3.748214	1
...	...	...
494	-4.422548	0
495	-3.563515	0
496	-4.729118	0
497	-4.766807	0
498	-2.527569	1

[499 rows x 6 columns]

Optization problem

Before we rank the solvents according to our target properties, we still need to define what we exactly mean with "optimal" solvent. Let's dive into the foundations of what "optimal" means from a mathematical point of view.

Definition

An **optimum** (plural: optima) is a solution that **maximizes or minimizes a given objective function** under specified constraints.

Let

$$\mathbf{x} = (x_1, x_2, \dots, x_n)^T \in \mathbb{R}^n$$

be the decision vector. This vector contains the decision variables that we aim to optimize.

The general form of an optimization problem is

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) \\ \text{s.t.} \quad & g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m, \\ & h_j(\mathbf{x}) = 0, \quad j = 1, \dots, p, \\ & \mathbf{x} \in \mathcal{X}, \end{aligned}$$

where:

- $f(\mathbf{x})$: is the objective function,
- $g_i(\mathbf{x}, \mathbf{y})$: inequality constraints, and
- $h_j(\mathbf{x}, \mathbf{y})$: equality constraints.

In simple terms: Our goal is to minimize (or maximize) the objective function. To reach that goal, we need to find the right values for the decision variables x_n . The constraints act as boundaries that define the search space in which we look for the right x_n .

Example:

- Finding the temperature that maximizes the yield in a chemical reaction
- Minimizing energy consumption in a processing

Pareto-Optimum (Pareto-Optimal Solution)

When there are **multiple objectives** that may conflict (multi-objective optimization), a single optimum often does not exist. Instead, we talk about **Pareto-optimal solutions**.

Definition:

A solution is **Pareto-optimal** if **no objective can be improved without worsening at least one other objective**.

- The set of all Pareto-optimal solutions is called the **Pareto front** or **Pareto frontier**.
- These solutions represent **trade-offs** between competing objectives.

Example:

- Maximizing yield **and** minimizing cost in a chemical process
 - Increasing yield may increase cost

- A Pareto-optimal solution balances the trade-off

Summary

- **Optimum** = best solution for a single goal
- **Pareto-optimum** = best compromise solutions when multiple goals conflict

Our example

Let's define the optimization problem for our example. First, we define our decision variables.

$$\mathbf{x} = (x_{sol}, x_{cost})^\top$$

where x_{sol}, x_{cost} in $\mathbb{R}^n$.

We define our objective function as a *Score* that describes how well our solvent performs in terms of solubility and cost. We will define the *Score* later. We aim to get a maximum *Score*, therefore, our optimization problem is defined as:

$$\max_{x_{sol}, x_{cost}} Score$$

We want all solvents to be non-toxic, therefore, we add this as equality constraint:

$$\text{s.t.} \quad y_{tox} = 0$$

,

with

$$y_{tox} \in \{0, 1\}$$

,

where 0 represents non-toxic solvents and 1 represents toxic solvents.

How do we solve our optimization problem?

1. We need to define the objective function that serves as a metric for our ranking.
2. We filter out all solvents that are toxic.
3. We need to rank the remaining solvents according to our objective function.

Objective function

How can we measure, how well a solvent performs? Let's have a look at the value ranges of our decision variables. When we want to rank the solvents, we must make sure that both objectives are on the same scale, preferably between 0 and 1:

Solubility:

The GNN predicts solubility values on a log scale $\log(x_{sol,GNN}) \in (-\infty, 0)$,

where $10^{x_{sol,GNN}}$ gives the actual solubility in linear scale.

We need to convert the log-scale to linear scale, otherwise the small solubility differences have a larger impact on the ranking (example: small solubilities on the log-scale such as -7 vs -8 give a difference of 1. We need to convert these values to linear scale: 10^{-7} - 10^{-8} give a difference of 0.00000009). After conversion, solubility is automatically scaled between 0 and 1.

$$x_{sol,norm} = 10^{\log(x_{sol,GNN})}$$

As we want to find a solvent with maximum lignin solubility, we aim at high values for $x_{sol,norm}$.

Cost: The cost is given as $x_{cost} \in (-\infty, 0]$. To scale the cost between 0 and 1, we need to set an upper limit. In our case, we simply choose the maximum price in the solvent list. Since we want to minimize the cost, we aim at lower values. You can see that we have conflicting objectives: We want to minimize the cost while maximizing solubility. Therefore, we need a reversal of the scale for one of the objectives, here we choose x_{cost} :

$$x_{cost,norm} = 1 - \frac{x_{cost}}{x_{cost,max}}$$

Finally, we can rank the solvents according to the score obtained based on the following objective function:

$$\begin{aligned} &\max_{x_{sol}, x_{cost}} Score \\ Score &= w_{sol} \cdot x_{sol,norm} + w_{cost} \cdot w_{cost,norm} \end{aligned}$$

, where

$$\sum_j w_j = 1$$

with $j \in \{sol, cost\}$.

In the objective function, we can add the weights w_j to scale the importance of the single objectives according to our desires.

Solving the optimization problem

Let's solve the optimization problem! First, we eliminate all solvents that violate our constraints:

Non-toxic solvents: 325
Toxic solvents: 174

	Solvent	Smiles	Cost	canonical_smiles \
0	Methyl Acetate	CC(OC)=O	0.020	COC(C)=O
1	Methyl-t-Butyl Ether	CC(C)(C)OC	0.020	COC(C)(C)C
2	Butyraldehyde	CCCC=O	0.034	CCCC=O
3	2-Methyl Tetrahydrofuran	CC1CCC01	0.111	CC1CCC01
4	Isobutyl Mercaptan	CC(CS)C	0.190	CC(C)CS
...	...	...	...	...
320	Octyl Mercaptan	CCCCCCCCS	0.048	CCCCCCCCS
321	Nonyl Mercaptan	CCCCCCCCCS	1.250	CCCCCCCCCS
322	Tert-Butyl Mercaptan	S[C@](C)(C)C	0.186	CC(C)(C)S
323	Tert-Butylbenzene	CC(C)(C1=CC=CC=C1)C	0.062	CC(C)(C)c1ccccc1
324	Methyl-t-Butyl Ether	CC(C)(C)OC	0.040	COC(C)(C)C

	GNN_prediction_lignin	acute_toxicity	x_sol_norm	x_cost_norm
0	-1.675693	0	0.021101	0.999760
1	-4.766807	0	0.000017	0.999760
2	-2.073424	0	0.008445	0.999592
3	-3.061253	0	0.000868	0.998669
4	-3.683444	0	0.000207	0.997722
...	...	...	...	...
320	-4.422548	0	0.000038	0.999424
321	-4.422548	0	0.000038	0.985012
322	-3.563515	0	0.000273	0.997770
323	-4.729118	0	0.000019	0.999257
324	-4.766807	0	0.000017	0.999520

[325 rows x 8 columns]

Now, our optimization problem simplifies to a very simple unconstrained linear program. We will now:

1. Rank the solvents according to our *Score*

2. Look at pareto-optimal solutions

	Solvent	Smiles	Cost	\
0	Dimethyl Sulfoxide (DMSO)	<chem>O=S(C)C</chem>	0.046	
1	N,N-Dimethyl Acetamide	<chem>CC(N(C)C)=O</chem>	0.032	
2	Dimethyl Sulfite	<chem>O(C)S(=O)OC</chem>	0.659	
3	Mango Furanone	<chem>CC1C(=O)C=C(O1)C</chem>	0.343	
4	1,3-Dioxolane	<chem>C1OCCO1</chem>	0.096	
...	...	...	...	
320	2-Methyl-2-Cyclopenten-1-one	<chem>CC1=CCCC1=O</chem>	36.223	
321	Cyclohexane, Methylene-	<chem>C=C1CCCCC1</chem>	34.560	
322	1,1-Dimethylcyclohexane	<chem>CC1(C)CCCC1</chem>	43.600	
323	Cyclobutanol	<chem>OC1CCC1</chem>	52.184	
324	Pinocarveol	<chem>CC1(C2CC1C(=C)C(C2)O)C</chem>	83.400	
	canonical_smiles	GNN_prediction_lignin	acute_toxicity	\
0	<chem>CS(C)=O</chem>	-0.352808	0	
1	<chem>CC(=O)N(C)C</chem>	-1.005784	0	
2	<chem>COS(=O)OC</chem>	-1.277906	0	
3	<chem>CC1=CC(=O)C(C)O1</chem>	-1.398501	0	
4	<chem>C1COCO1</chem>	-1.451894	0	
...	...	...	...	
320	<chem>CC1=CCCC1=O</chem>	-1.581633	0	
321	<chem>C=C1CCCCC1</chem>	-4.987967	0	
322	<chem>CC1(C)CCCC1</chem>	-6.353435	0	
323	<chem>OC1CCC1</chem>	-2.083833	0	
324	<chem>C=C1C(O)CC2CC1C2(C)C</chem>	-2.682380	0	
	x_sol_norm	x_cost_norm	Score	
0	4.438044e-01	0.999448	0.721626	
1	9.867698e-02	0.999616	0.549147	
2	5.273445e-02	0.992098	0.522416	
3	3.994835e-02	0.995887	0.517918	
4	3.532694e-02	0.998849	0.517088	
...	...	...	...	
320	2.620399e-02	0.565671	0.295938	
321	1.028094e-05	0.585612	0.292811	
322	4.431650e-07	0.477218	0.238609	
323	8.244547e-03	0.374293	0.191269	
324	2.077877e-03	0.000000	0.001039	

[325 rows x 9 columns]

Nice! For this ranking we used the objective function and maximized it. Now, we have a list of the candidates and their *Score*. While this ranking is very nice in giving us interesting candidates, it cannot very well represent trade-offs. Therefore, we take a look at the pareto front.

Recap:

A **Pareto front** represents the set of solutions where **no objective can be improved without sacrificing at least one other objective**.

We can find pareto-optimal solutions by dominance.

- **Dominance:**
A solution A *dominates* solution B if:
 1. A is at least as good as B in all objectives, and
 2. A is strictly better than B in at least one objective.

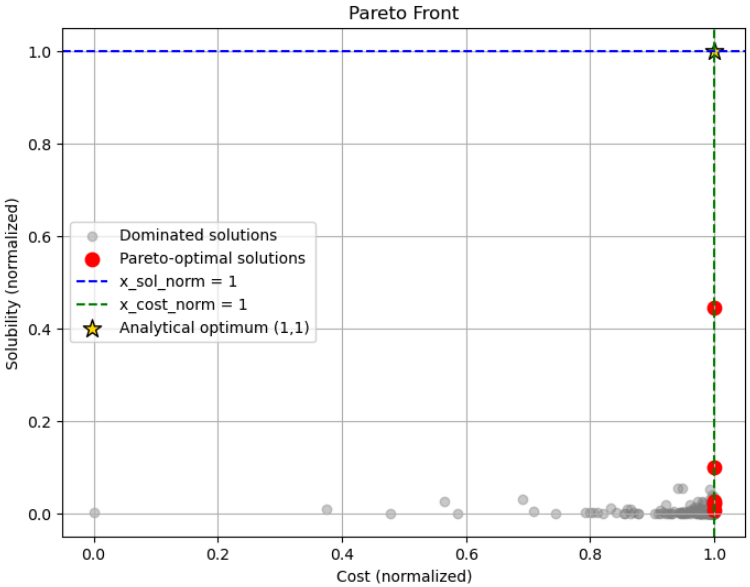
In simple words, a pareto-optimal solution is a solution that **is not dominated by any other solution**.

If we had values across both objectives on the whole range between 0 and 1, the pareto fron would look like this:

	Solvent	Smiles	Cost	canonical_smiles	\
0	Dimethyl Sulfoxide (DMSO)	O=S(C)C	0.046	CS(C)=O	
1	N,N-Dimethyl Acetamide	CC(N(C)C)=O	0.032	CC(=O)N(C)C	
7	γ-Butyrolactone (GBL)	O=C1CCC01	0.025	O=C1CCC01	
15	Methyl Acetate	CC(OC)=O	0.020	COC(C)=O	
61	1-Butanol	CCCCO	0.019	CCCCO	

	GNN_prediction_lignin	acute_toxicity	x_sol_norm	x_cost_norm	Score	\
0	-0.352808	0	0.443804	0.999448	0.721626	
1	-1.005784	0	0.098677	0.999616	0.549147	
7	-1.587462	0	0.025855	0.999700	0.512777	
15	-1.675693	0	0.021101	0.999760	0.510431	
61	-2.264847	0	0.005434	0.999772	0.502603	

	is_pareto
0	True
1	True
7	True
15	True
61	True



Discussion

Solvent candidates

Let's have a look at the solvent candidates that we identified in the ranking. This is our Top 10:

	Solvent	canonical_smiles	GNN_prediction_lignin	Cost
0	Dimethyl Sulfoxide (DMSO)	CS(C)=O	-0.352808	0.046
1	N,N-Dimethyl Acetamide	CC(=O)N(C)C	-1.005784	0.032
2	Dimethyl Sulfite	COS(=O)OC	-1.277906	0.659
3	Mango Furanone	CC1=CC(=O)C(C)O1	-1.398501	0.343
4	1,3-Dioxolane	C1COCO1	-1.451894	0.096
5	Acetol	CC(=O)CO	-1.477099	0.118
6	Acetone	CC(C)=O	-1.556779	0.035
7	γ-Butyrolactone (GBL)	O=C1CCC01	-1.587462	0.025
8	Caprolactone (Epsilon)	O=C1CCCCC01	-1.577711	0.084

Now we have a look at the pareto-optimal candidates:

	Solvent	GNN_prediction_lignin	Cost
0	Dimethyl Sulfoxide (DMSO)	-0.352808	0.046
1	N,N-Dimethyl Acetamide	-1.005784	0.032
7	γ-Butyrolactone (GBL)	-1.587462	0.025
15	Methyl Acetate	-1.675693	0.020
61	1-Butanol	-2.264847	0.019

Although we were already considering acute toxicity, lignin solubility and cost, it's still important to get a clear picture about the environmental, health and safety risks of these solvents as well as solvent properties that impact the process, such as the melting temperature (T_m), boiling temperature at atmospheric pressure (T_b), and the enthalpy of vaporization $\Delta H_{vap}(at T_b)$:

Solvent	CHEM21 category	Acute toxicity (GHS)	Other toxicity / EHS issues	T_m (°C)	T_b (°C)	ΔH_{vap} (kJ/mol)
Dimethyl sulfoxide (DMSO)	Problematic	Not classified as acutely toxic	Strong skin penetration enhancer; eye/skin irritation	18.5	189	~56
N,N-Dimethylacetamide (DMAc)	Hazardous	Acute Tox. 4	Reprotoxic (Cat. 1B); hepatotoxic; skin absorption	-20	165	~54
γ-Butyrolactone (GBL)	Not listed	Acute Tox. 4	CNS effects; regulatory restrictions (GHB precursor)	-43	204	~50
Methyl acetate	Recommended or problematic	Not classified as acutely toxic	Highly flammable; narcotic effects at high exposure	-98	57	~31
1-Butanol	Recommended	Acute Tox. 4	Eye/respiratory irritation; CNS effects	-89	118	~43
Dimethyl sulfite	Not listed	Not classified as acutely toxic	Respiratory tract irritation, easily hydrolyzes	-50	126	~35
Mango furanone	Not listed	Not classified as acutely toxic	Data sparse; food flavoring; low exposure data	~30	~170	~45
1,3-Dioxolane	Not listed	Acute Tox. 4	Flammable; peroxide former; irritation	-95	75	~28
Acetol (hydroxyacetone)	Not listed	Not classified (limited data)	Eye/skin irritation; hygroscopic; data gaps	-20	145	~45
Acetone	Recommended	Not classified as acutely toxic	Highly flammable; narcotic effects at high exposure, highly volatile, reactive	-95	56	~31
ε-Caprolactone	Not listed	Not classified as acutely toxic	Low toxicity; biodegradable; mild irritation	-43	238	~60

- DMSO is commonly used as a solvent for lignin dissolution, e.g. for nanoparticle formation. Our analysis shows suitable cost while having the highest lignin solubility of all analysed solvent candidates
- DMAc has low acute toxicity but is reprotoxic and hepatotoxic, rated as hazardous in the CHEM21 solvent selection guide and other
- GBL has low acute toxicity but effects on the central nervous systems
- Methyl acetate has low lignin solubility but low cost. Is recommended in the CHEM21 solvent selection guide
- Dimethyl sulfite is not inert and easily hydrolyzes in presence of water, not suitable for biomass which usually contains residual moisture
- Mango furanone is solid at room temperature
- 1,3-Dioxolane has low acute toxicity, but is very flammable and prone to peroxide formation
- Acetol strongly reacts with acids and bases
- ε-Caprolactone is a non-toxic, biodegradable solvent, but has a high boiling point

You can see, there are a lot of properties apart from acute toxicity, lignin solubility and cost that need to be taken into account for solvent selection. Let's focus on the four most promising candidates and perform an in-depths comparison.

First, we merge both dataframes...

	Solvent	canonical_smiles	x_sol_norm	x_cost_norm
0	Dimethyl Sulfoxide (DMSO)	CS(C)=O	0.443804	0.999448
1	N,N-Dimethyl Acetamide	CC(=O)N(C)C	0.098677	0.999616
2	Dimethyl Sulfite	COS(=O)OC	0.052734	0.992098
3	Mango Furanone	CC1=CC(=O)C(C)O1	0.039948	0.995887
4	1,3-Dioxolane	C1COC01	0.035327	0.998849
5	Acetol	CC(=O)CO	0.033335	0.998585
6	Acetone	CC(C)=O	0.027747	0.999580
7	Î³-Butyrolactone (GBL)	O=C1CCC01	0.025855	0.999700
8	Caprolactone (Epsilon)	O=C1CCCCC01	0.026442	0.998993
9	Methyl Acetate	COC(C)=O	0.021101	0.999760
10	1-Butanol	CCCCO	0.005434	0.999772

... and then we delete all candidates that we consider to be unsuitable based on our analysis:

	Solvent	canonical_smiles	x_sol_norm	x_cost_norm
0	Dimethyl Sulfoxide (DMSO)	CS(C)=O	0.443804	0.999448
1	Mango Furanone	CC1=CC(=O)C(C)O1	0.039948	0.995887
2	Acetone	CC(C)=O	0.027747	0.999580
3	Caprolactone (Epsilon)	O=C1CCCCC01	0.026442	0.998993
4	Methyl Acetate	COC(C)=O	0.021101	0.999760
5	1-Butanol	CCCCO	0.005434	0.999772

Now we add further information to the table: Melting points, boiling points and enthalpy of evaporation. For only a handful of solvents, it's managable to retrieve this data manually from public databases:

Good sources for searching this kind of data are:

- [PubChem](#)
- [NIST Webbook](#)

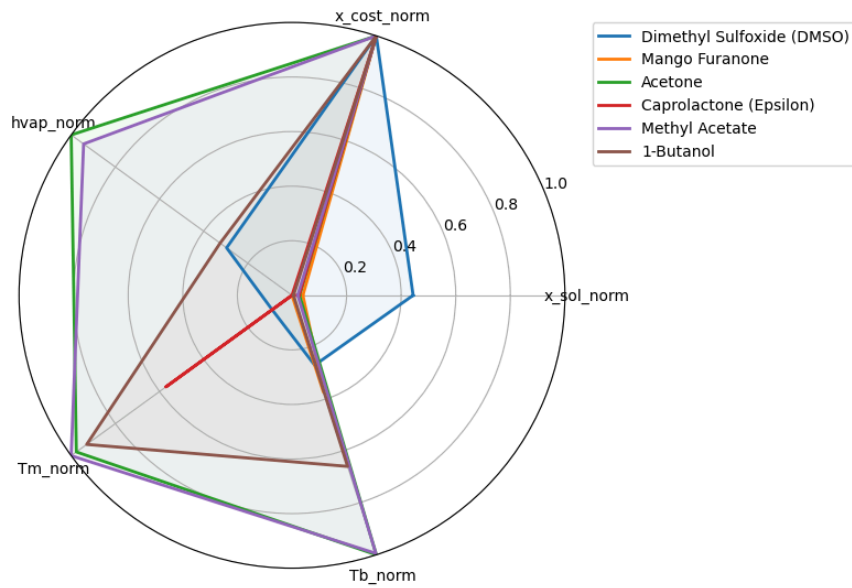
For our example, I copy-pasted the values from these databases

... and normalized the values. During the normalization procedure, I again switched the sorting: we want minimum enthalpy of vaporization and also low boiling points. Therefore, I reversed the order, so that 1 indicates that this target is met and 0 means it's not met. For the melting point, we don't have a preference. I still applied the same procedure to the melting points for consistency.

	Solvent	canonical_smiles	x_sol_norm	x_cost_norm	\
0	Dimethyl Sulfoxide (DMSO)	CS(C)=O	0.443804	0.999448	
1	Mango Furanone	CC1=CC(=O)C(C)O1	0.039948	0.995887	
2	Acetone	CC(C)=O	0.027747	0.999580	
3	Caprolactone (Epsilon)	O=C1CCCCC01	0.026442	0.998993	
4	Methyl Acetate	COC(C)=O	0.021101	0.999760	
5	1-Butanol	CCCCO	0.005434	0.999772	

	hvap	Tm	Tb	hvap_norm	Tm_norm	Tb_norm
0	52.90	18.5	189	0.296128	0.089844	0.269231
1	NaN	30.0	170	NaN	0.000000	0.373626
2	31.27	-95.0	56	1.000000	0.976562	1.000000
3	62.00	-43.0	238	0.000000	0.570312	0.000000
4	33.00	-98.0	57	0.943703	1.000000	0.994505
5	52.00	-89.0	118	0.325415	0.929688	0.659341

Spider Plot of Solvent Properties (Normalized)



The plot shows us that:

- DMSO is the solvent that maximizes lignin solubility among the candidates, while also having a competitive price. However, for processes that require low boiling and melting points, DMSO is not the right choice.
- All other solvents have a dramatically lower lignin solubility.
- Especially acetone and methyl acetate are easily volatile, and could be interesting candidates for processes where evaporation is important.

There is no solvent that dominates all other solvents. We see that without defining the process explicitly, we cannot say what is "the optimal solvent". However, we found many solvents that are interesting candidates. By using computational models, we were able to narrow down the number of solvents that we now would need to test in the lab. These models accelerated the search and also brought us to solvent candidates that we wouldn't have considered from "pure intuition".